

Review of *Changing Base without Losing Space*

Advanced Algorithms Paper Review

July 15, 2026

Abstract

This review discusses the paper *Changing Base without Losing Space* by Yevgeniy Dodis, Mihai Pătraşcu, and Mikkell Thorup. The paper studies how to change the representation base of data without losing space while preserving locality. Its two main results are an optimal-space vector representation supporting constant-time reads and writes, and an online prefix-free encoding scheme with near-optimal redundancy and logarithmic memory.

Contents

1	Introduction	4
1.1	Paper Overview	4
1.2	The Two Main Problems	4
1.3	Main Results	4
1.4	Why the Results Are Surprising	5
1.5	Technical Roadmap	5
2	Problem and Motivation	5
2.1	Vector Representation as a Succinct Data Structure Problem	5
2.2	Global Base Conversion and the Loss of Locality	6
2.3	Naive Local Encoding and Linear Redundancy	6
2.4	Redundancy in Succinct Representations	7
2.5	The Role of the Word RAM Model	7
2.6	Why Arithmetic Coding Is Not Enough	8
2.7	Online Prefix-Free Encoding as a Second Motivation	8
2.8	Why Elias Codes Are Not Enough	9
2.9	The Naive Block-Marker Method and Linear Redundancy	9
2.10	From EOF Markers to Base Conversion	9
2.11	Theorem 2 and Its Significance	10
2.12	Cryptographic Motivation: Cascade Constructions	11
2.13	Examples of Extension Attacks	12
2.14	Motivational Summary for the Second Problem	12
2.15	A Unified View of the Two Problems	13

3	The SOLE Encoding	13
3.1	Problem Setting	13
3.2	The First Pass: Short-Odd Long-Even	13
3.3	Regrouping and the Second Pass	14
3.4	Termination and Prefix-Freeness	14
3.5	A Concrete Example	14
3.6	Locality and Updates	15
3.7	Variants and Practical Remarks	15
3.8	From SOLE to Information Carriers	15
4	Information Carriers	16
4.1	The Spill Problem	16
4.2	The Information Carrier Abstraction	16
4.3	The Information Carrier Lemma	16
4.4	Construction	17
4.5	Decoding and Correctness	17
4.6	Redundancy Analysis	17
4.7	Constant-Time Implementation	18
4.8	Why the Lemma Is Powerful	18
5	Optimal-Space Vector Representation	18
5.1	The Representation Problem	18
5.2	Grouping Symbols into Carriers	19
5.3	A Sequential First Attempt	19
5.4	Redundancy of the Sequential Construction	20
5.5	Constant-Time Reads and Writes	20
5.6	Why the Sequential Construction Is Insufficient	20
5.7	The Implicit Binary-Tree Representation	21
5.8	Queries and Updates in the Tree	21
5.9	Reducing the Number of Precomputed Constants	21
5.10	The Final-Bit Issue	22
5.11	The Vector Representation Theorem	22
6	Online Prefix-Free Encoding	22
6.1	The Online Universal-Coding Problem	22
6.2	Slowly Growing Blocks	23
6.3	Handling the Final Partial Block	23
6.4	Pass One: Embedding an Early EOF Symbol	23
6.5	Redundancy of Alphabet Enlargement	24
6.6	Pass Two: Conversion through Information Carriers	24
6.7	Why the Decoder Stops in Time	25
6.8	Total Encoding Length	25
6.9	Memory and Running Time	25
6.10	Significance of the Construction	25
7	Related Work	26
7.1	Succinct Data Structures and Redundancy	26
7.2	Rank, Select, and Lower Bounds	26
7.3	Arithmetic Coding and Locality	26

7.4	Universal and Prefix-Free Coding	27
7.5	Cryptographic Domain Extension	27
7.6	Subsequent Developments after 2010	28
7.7	Positioning the Paper	28
8	Correctness and Complexity Analysis	28
8.1	Proof Architecture	28
8.2	Correctness of the SOLE Transform	29
8.3	Correctness and Redundancy of the Information-Carrier Lemma	30
8.4	Proof of the Optimal-Space Vector Result	31
8.5	Proof of the Online Prefix-Free Result	32
8.6	Computational Model and Complexity Assumptions	33
8.7	Scope of the Guarantees	33
9	Strengths, Limitations, and Open Questions	33
9.1	Strengths	33
9.2	Limitations	34
9.3	Open Questions	35
10	Conclusion	36
10.1	Summary	36
10.2	The Unifying Role of Information Carriers	36
10.3	What Makes the Paper Surprising	37
10.4	Theoretical Value	37
10.5	Practical Applicability	37
10.6	Final Assessment	37

1 Introduction

1.1 Paper Overview

Changing Base without Losing Space, by Yevgeniy Dodis, Mihai Pătraşcu, and Mikkel Thorup, studies how to represent data from an arbitrary finite alphabet in binary memory without sacrificing either space or locality. The difficulty is elementary but pervasive: the natural number of states of a data item is often not a power of two. Rounding every item to an integral number of bits preserves local access but repeatedly wastes space, whereas encoding the entire data set as one integer is space-optimal but global.

The paper develops local, reversible base-conversion techniques that avoid this apparent trade-off. The same technical idea supports both a succinct random-access data structure and a low-memory online code.

1.2 The Two Main Problems

The first problem is to represent a vector

$$A[1..n]$$

whose entries come from an alphabet of size Σ . There are Σ^n possible vectors, so the exact information-theoretic target is

$$\lceil n \log_2 \Sigma \rceil$$

bits. The representation must nevertheless support reading and changing an arbitrary entry in constant time.

The second problem is online prefix-free encoding. An input bit stream has an initially unknown length n , and its encoding must be self-delimiting while using little working memory. Treating termination as a separate marker bit on every block is online but wasteful; writing the length first is compact but requires knowing the future input. Prefix-freeness is also important for variable-length cascade constructions in cryptography, where an encoded message must not be extendable into another valid encoded message.

1.3 Main Results

On the Word RAM, the paper represents a vector over $[\Sigma]$ in exactly

$$\lceil n \log_2 \Sigma \rceil$$

bits while supporting both reads and writes in $O(1)$ time. Thus the representation has no redundancy beyond the unavoidable final rounding to an integral number of bits.

For the streaming problem, the paper constructs an online universal code of length

$$n + \log_2 n + O(\log \log n).$$

Encoding and decoding use $O(\log n)$ bits of memory and take constant time per word.

1.4 Why the Results Are Surprising

Standard approaches suggest that optimal space and locality should conflict. A mixed-radix or arithmetic-style encoding can approach the entropy bound, but one symbol may influence a large part of the representation. Independent fixed-width fields give immediate access but incur a rounding loss for every entry. Similarly, classical self-delimiting codes attain small overhead only after placing length information before the data, while simple online marker schemes pay repeatedly. The two results show that these losses come from the chosen representations rather than from inherent lower bounds.

1.5 Technical Roadmap

The paper first presents the Short-Odd Long-Even (SOLE) encoding, a concrete local conversion from an alphabet enlarged by an end-of-file symbol back to ordinary binary blocks. It then abstracts SOLE’s temporary non-binary values as *spills* and shows how a sufficiently large information carrier can commit an old spill to memory while leaving only a much smaller new spill.

The information-carrier lemma is subsequently applied in two directions. Grouped vector entries and a carefully organized representation yield exact space with local reads and updates. Slowly growing streaming blocks use the same mechanism to absorb termination information with near-optimal overhead. The following sections first make the two motivating problems precise and then develop SOLE and the information-carrier abstraction.

2 Problem and Motivation

2.1 Vector Representation as a Succinct Data Structure Problem

The first main problem studied in the paper is the representation of a vector

$$A[1..n],$$

where each entry $A[i]$ belongs to a finite alphabet Σ . As in the paper, we use Σ both for the alphabet itself and, when the meaning is clear, for its cardinality.

There are Σ^n possible vectors of length n . Therefore, any representation that distinguishes all such vectors must use at least

$$\log_2(\Sigma^n) = n \log_2 \Sigma$$

bits of information. Since an actual binary representation must use an integral number of bits, the information-theoretic optimum is

$$\lceil n \log_2 \Sigma \rceil$$

bits.

This lower bound is easy to state, but it is not easy to achieve together with efficient operations. The data structure must not only store the vector using optimal space, but also support reading and writing an arbitrary entry $A[i]$ in constant time. This is why the problem is a basic example in succinct data structures: it asks whether one can simultaneously achieve information-theoretic optimality and local access.

2.2 Global Base Conversion and the Loss of Locality

A direct way to achieve the optimal space bound is to view the entire vector as one large integer. More precisely, the sequence

$$A[1], A[2], \dots, A[n]$$

can be interpreted as a number in the range

$$\{0, 1, \dots, \Sigma^n - 1\}.$$

Writing this number in binary gives a representation using exactly

$$\lceil n \log_2 \Sigma \rceil$$

bits.

The problem is that this representation is global. A single entry $A[i]$ is not stored in a fixed, independently addressable memory region. Recovering it generally requires arithmetic on the whole encoded number, and modifying it may affect the binary representation in a non-local way. Thus, although global base conversion proves that the optimal space is achievable in principle, it does not give a useful local data structure.

This distinction is central to the paper. The goal is not merely to compress a vector into the fewest possible bits. The goal is to perform base conversion while preserving locality.

2.3 Naive Local Encoding and Linear Redundancy

The opposite approach is to encode each entry separately. Since each entry has Σ possible values, one can store every entry using

$$\lceil \log_2 \Sigma \rceil$$

bits. Then the position of $A[i]$ is easy to compute, and both reading and writing are local operations.

However, this representation uses

$$n \lceil \log_2 \Sigma \rceil$$

bits in total. When Σ is not a power of two, this is larger than the information-theoretic optimum. The redundancy is

$$n \lceil \log_2 \Sigma \rceil - n \log_2 \Sigma.$$

Although the loss per entry is less than one bit, it is repeated n times and therefore becomes linear.

For example, when $\Sigma = 10$, each decimal digit contains only

$$\log_2 10 \approx 3.322$$

bits of information. But storing one decimal digit in a fixed-width binary field requires

$$\lceil \log_2 10 \rceil = 4$$

bits. Thus, each digit wastes approximately

$$4 - \log_2 10 \approx 0.678$$

bits, and a vector of n decimal digits wastes about $0.678n$ bits.

This illustrates the basic tension. Global encoding is space-optimal but non-local. Local fixed-width encoding is easy to access but wastes linear space. The main result of the paper shows that this tension is not inherent.

2.4 Redundancy in Succinct Representations

In succinct data structures, the space usage of a representation is often compared with the information-theoretic minimum. The difference between the actual space and the minimum is called redundancy. For the vector representation problem, the target is therefore not merely

$$n \log_2 \Sigma + o(n)$$

bits, but ideally the exact bound

$$\lceil n \log_2 \Sigma \rceil.$$

Before this paper, known techniques could support constant-time access while using space of the form

$$n \log_2 \Sigma + \text{redundancy}.$$

The redundancy had been reduced to lower-order terms, such as $O(n/\log n)$, $O(n/\log^2 n)$, or more generally $n/\log^{O(1)} n$ in suitable settings. These results already showed that the naive linear redundancy was not necessary.

Theorem 1 goes further. It removes the redundancy entirely. On the Word RAM, the paper shows that one can represent the vector using exactly

$$\lceil n \log_2 \Sigma \rceil$$

bits while supporting both reading and writing any entry in $O(1)$ time.

The write operation is particularly important. A representation that only supports queries might still hide a highly global encoding. Supporting updates means that changing one entry must be handled locally, without propagating changes through the whole representation.

2.5 The Role of the Word RAM Model

The paper states its result in the Word RAM model. In this model, memory is randomly addressable and divided into words of w bits, where

$$w = \Omega(\log n).$$

This assumption ensures that an index or pointer can fit into a constant number of words.

The paper uses standard word-level arithmetic operations, including addition, multiplication, and division, as constant-time operations. This matters because the later constructions are based on reversible arithmetic transformations. They repeatedly combine values, split them into quotients and remainders, and move information between ranges whose sizes are not powers of two.

Thus, the constant-time guarantee in Theorem 1 should be understood as a Word RAM guarantee. The construction is not merely a bit-by-bit coding argument; it uses word-level arithmetic to obtain local base conversion.

2.6 Why Arithmetic Coding Is Not Enough

Arithmetic coding is the standard information-theoretic method for converting between alphabets. Given a distribution D , it can encode a sequence of n symbols using about

$$nH(D) + O(1)$$

bits in expectation, where $H(D)$ is the binary entropy of D . For the uniform distribution over an alphabet of size Σ , this becomes

$$n \log_2 \Sigma + O(1).$$

At first glance, arithmetic coding seems to solve the same space problem as the paper. It can represent symbols from a non-binary alphabet in nearly optimal binary space. However, arithmetic coding does not provide the worst-case locality required for a data structure.

The reason is that arithmetic coding represents the whole sequence by an interval. Each symbol refines the current interval, and the final output is a binary number lying inside the final interval. Consequently, the output bits are not naturally partitioned by input position. The representation of one symbol may depend on many surrounding symbols.

This non-locality is also visible in bounded-precision implementations. Such implementations may delay output when the current interval does not yet determine the next bit. The encoder then maintains a number of outstanding bits and later emits a burst of bits once the interval becomes sufficiently separated. This bursty behavior is incompatible with worst-case local access.

Therefore, arithmetic coding is an excellent global compression technique, but it is not a solution to the vector representation problem considered in this paper. The paper needs a different kind of base conversion: one that is reversible, local, and compatible with constant-time reads and writes.

2.7 Online Prefix-Free Encoding as a Second Motivation

The second main problem studied in the paper is online prefix-free encoding. Suppose a stream of n bits arrives online, where the final length n is not known in advance. The goal is to produce an encoding that is prefix-free: no valid encoded message should be a prefix of another valid encoded message. Equivalently, the encoded message should be self-delimiting, so that the decoder can determine where the message ends without receiving the length n separately.

In information theory, this task is closely related to universal coding. In computer science and cryptography, the same requirement is often described using the terms prefix-free code or self-delimiting code. The important point in this paper is that the encoding must be produced online and with very small memory. The encoder cannot first read the whole input, compute its length, and then go back to write the length at the front. Such a solution would require storing the entire input stream, which is incompatible with the desired $O(\log n)$ -bit memory bound.

Thus, the problem is not merely to represent an unknown-length string compactly. The problem is to do so while satisfying three simultaneous constraints:

prefix-freeness, online processing, low redundancy and low memory.

2.8 Why Elias Codes Are Not Enough

The textbook solution for universal coding is given by Elias codes. For example, Elias delta-code encodes the length n first and then appends the n bits of data. This gives an encoding length of

$$n + O(\log n)$$

bits. By applying the same idea recursively, Elias omega-code obtains a length of the form

$$n + \log_2 n + \log_2 \log_2 n + \cdots + O(\log^* n),$$

which is optimal up to lower-order terms.

However, Elias codes are inherently offline for the purpose of this paper. Their structure requires the length information to be written before the data. In the online setting, the encoder does not know n when the first input bits arrive. If it waits until the end of the stream to learn n , it must either buffer the entire input or output the length after the data, which does not give the usual self-delimiting structure. Therefore, Elias codes show what overhead is information-theoretically possible, but they do not solve the low-memory online encoding problem considered in the paper.

2.9 The Naive Block-Marker Method and Linear Redundancy

A simple online solution is to divide the input into fixed-size blocks. Suppose each block has b bits. The encoder can output each block immediately, and append one extra marker bit indicating whether this block is the last block. This method is online and easy to decode.

The cost is that one extra bit is paid for every b input bits. For an n -bit input, the redundancy is therefore

$$\Theta(n/b).$$

When b is fixed, this is linear in n . Even if b is large in practice, for example $b = 128$, the scheme still wastes a constant fraction of the stream size. The paper emphasizes that this is too high a price to pay merely for encoding where the stream ends.

This naive construction also suggests a misleading trade-off: to reduce redundancy, one must increase the block size, but increasing the block size means buffering more input. This gives the impression that small-space online prefix-free encoding may necessarily require linear redundancy. Maurer and Sjödin conjectured such a limitation in the small-space online setting. The paper refutes this intuition by showing that the apparent trade-off comes from thinking in terms of integral bits.

2.10 From EOF Markers to Base Conversion

The key observation is that an end-of-file marker should not be treated as one full additional bit per block. Let

$$B = 2^b.$$

A normal b -bit block is an element of

$$[B] = \{0, 1, \dots, B - 1\}.$$

If we add one special end-of-file symbol `eof`, the alphabet size becomes

$$B + 1$$

rather than $2B$. The additional information per symbol is therefore

$$\log_2(B+1) - \log_2 B = \log_2 \left(1 + \frac{1}{B}\right),$$

which is much smaller than one bit when B is large.

Thus, the real problem is not how to attach a separate marker bit to each block. The real problem is a local base-conversion problem:

convert a stream over an alphabet of size $B+1$ back into B -ary blocks.

If this conversion can be done online, locally, and with low redundancy, then prefix-freeness can be obtained without paying one bit per block.

The paper illustrates this idea with a simple estimate. Suppose the stream contains n blocks of b bits and assume temporarily that

$$n \leq 2^b.$$

After adding an EOF symbol, one needs to encode $n+1$ symbols from an alphabet of size 2^b+1 . An ideal base conversion would use

$$(n+1) \log_2(2^b+1)$$

bits. Since the original data contains nb bits, the overhead is

$$(n+1) \log_2(2^b+1) - nb.$$

Using

$$\log_2(2^b+1) = b + \log_2 \left(1 + \frac{1}{2^b}\right),$$

this overhead becomes roughly

$$b + n \log_2 \left(1 + \frac{1}{2^b}\right).$$

Under the assumption $n \leq 2^b$, this is only

$$b + O(1).$$

The remaining challenge is to realize this conversion efficiently and locally. Section 2 begins with the SOLE encoding as a simple construction for this block setting, and Section 5 later develops the full online prefix-free code for binary streams.

2.11 Theorem 2 and Its Significance

SOLE explains the local conversion idea in a simplified block setting. The full result for online binary streams is stated in the paper's second main theorem: there exists a universal code mapping n input bits to

$$n + \log_2 n + O(\log \log n)$$

bits, while both encoding and decoding use only

$$O(\log n)$$

bits of memory and take constant time per operation.

This result is significant because it combines properties that are usually achieved separately. Elias codes give near-optimal length but are not suitable for the online low-memory setting. The naive block-marker method is online but incurs linear redundancy. The construction in this paper is both online and near-optimal in space. It shows that small-space online prefix-free encoding does not require a linear overhead.

2.12 Cryptographic Motivation: Cascade Constructions

Cryptographic constructions for messages of varying lengths also require prefix-free encoding. Many such constructions first define a compression function

$$f : \{0, 1\}^s \times \{0, 1\}^b \rightarrow \{0, 1\}^s,$$

where b is the block length and s is the state or buffer length. A long message

$$M = m_1 m_2 \cdots m_n$$

is split into b -bit blocks and processed by the cascade mode:

$$x_0 = IV,$$

and for $i = 1, \dots, n$,

$$x_i = f(x_{i-1}, m_i).$$

The final output is

$$y = x_n.$$

This general pattern appears in many settings, including CBC-MAC, cascade pseudo-random functions, Merkle–Damgård hash functions, and domain extension for message authentication codes. The security issue is that cascade constructions naturally expose an ancestor-descendant relation between messages. If

$$M_2 = M_1 M',$$

then

$$\text{Cascade}(M_2, f, IV) = \text{Cascade}(M', f, \text{Cascade}(M_1, f, IV)).$$

Thus, the output on a shorter message can become the internal state for computing the output on a longer message.

This is precisely the structure exploited by extension attacks. If valid messages are encoded in a prefix-free way, then no valid message can be a prefix of another valid message. Therefore, the cascade computation for two different valid messages does not place one message as an ancestor of the other in the computation tree. Prefix-free encoding is consequently a natural and general defense against extension attacks.

2.13 Examples of Extension Attacks

For CBC-MAC, the compression function has the form

$$f_k(x, m) = g_k(x \oplus m),$$

where g_k is a block cipher under key k . The CBC-MAC of a one-block message m_1 is

$$y = \text{CBC-MAC}(k, m_1) = g_k(m_1).$$

If messages are not encoded in a prefix-free way, an attacker can define

$$m_2 = m_1 \oplus y.$$

Then

$$\text{CBC-MAC}(k, m_1 m_2) = g_k(m_2 \oplus y) = g_k((m_1 \oplus y) \oplus y) = g_k(m_1) = y.$$

Thus, after learning the MAC of m_1 , the attacker can forge the MAC of a longer message $m_1 m_2$. Prefix-free encoding prevents this kind of confusion between messages of different lengths.

For cascade PRFs, suppose

$$F_k(M) = \text{Cascade}(M, f, k).$$

If $M_2 = M_1 M'$, then

$$F_k(M_2) = F_{F_k(M_1)}(M').$$

Thus, knowing the PRF value on an ancestor message allows the adversary to compute values on descendant messages, contradicting the intended pseudorandomness. Prefix-free encoding ensures that valid messages do not have this ancestor-descendant relation.

For Merkle–Damgård hash functions, the same structural problem appears as length extension. If

$$y = H(M),$$

then the cascade structure allows one to continue from state y and compute a value corresponding to an extended message. In an ideal random oracle, knowing $H(M)$ should not allow one to compute a related value for an extension of M . Prefix-free encoding gives a simple way to avoid this problem by ensuring that a completed valid message cannot be extended into another valid message.

2.14 Motivational Summary for the Second Problem

The online prefix-free encoding problem is therefore motivated from two directions. From the data representation perspective, it asks how to encode an unknown-length stream without paying a linear number of marker bits. From the cryptographic perspective, it provides a clean way to make cascade-based constructions safe for variable-length messages.

The paper’s key conceptual move is to reinterpret EOF handling as a base-conversion problem. Adding an EOF symbol enlarges the alphabet from B to $B+1$, but it should not force the representation to pay one full bit per block. The later constructions implement this idea and ultimately obtain overhead

$$\log_2 n + O(\log \log n).$$

2.15 A Unified View of the Two Problems

The two main problems of the paper have the same underlying form. In the vector representation problem, the alphabet size Σ need not be a power of two. In the online prefix-free encoding problem, adding an end-of-file symbol enlarges the block alphabet from B to $B + 1$. In both cases, the data naturally lives in ranges whose sizes do not match binary memory exactly.

There are two naive ways to handle this mismatch. One can encode everything globally as a large integer, which preserves optimal space but loses locality. Or one can round each local range up to the next power of two, which preserves locality but creates repeated rounding loss and therefore linear redundancy.

The contribution of the paper is to show that this apparent trade-off is avoidable. The authors develop local, reversible base-conversion techniques that move information between nearby blocks instead of encoding the whole object globally or rounding every symbol independently. The two problems therefore share the same underlying difficulty: data naturally belongs to ranges whose sizes are not powers of two, while machine memory is binary. The next section introduces SOLE encoding, the paper's first concrete construction showing that this mismatch can be handled locally and with very small redundancy.

3 The SOLE Encoding

3.1 Problem Setting

Let a machine block contain b bits, and define

$$B = 2^b, \quad [B] = \{0, 1, \dots, B - 1\}.$$

An ordinary block is an element of $[B]$. To make a stream self-delimiting, SOLE adds a distinguished symbol `eof`, identified with the value B ; the enlarged alphabet is therefore $[B + 1]$.

Basic SOLE considers an odd number n of input blocks and assumes

$$b \geq 2 \log_2 n + 1, \quad B \geq 2n^2.$$

Under these assumptions it encodes the n data blocks, together with the termination information, as $n + 2$ ordinary blocks. Its name abbreviates *Short-Odd Long-Even*: the first pass deliberately creates alternating short and long ranges, which the second pass recombines.

3.2 The First Pass: Short-Odd Long-Even

For $i = 0, 1, \dots$, the first pass processes symbols in positions $2i + 1$ and $2i + 2$. If their values are $a_0, a_1 \in [B + 1]$, it combines them as

$$z = a_1(B + 1) + a_0.$$

It then divides z by $B - 4i$, writing

$$z = q(B - 4i) + r.$$

The output pair is (r, q) , and the range conversion is

$$[B + 1]^2 \longrightarrow [B - 4i] \times [B + 4i + 4].$$

The parameter assumptions ensure the capacity inequality

$$(B + 1)^2 \leq (B - 4i)(B + 4i + 4).$$

Consequently $r \in [B - 4i]$ and $q \in [B + 4i + 4]$. Quotient and remainder uniquely determine z , and division by $B + 1$ then recovers a_0 and a_1 , so the transformation is reversible.

3.3 Regrouping and the Second Pass

After the first pass the ranges alternate between values slightly below and slightly above B . The second pass shifts the pairing by one position, so that a long value is coupled with the following short value. A typical pair has the conversion

$$[B + 4i] \times [B - 4i] \longrightarrow [B]^2.$$

Indeed,

$$(B + 4i)(B - 4i) = B^2 - 16i^2 < B^2.$$

Combining the two nonstandard values into one mixed-radix integer and taking its quotient and remainder modulo B therefore produces two ordinary blocks. The short and long ranges need not fit binary memory separately; their product fits once they are regrouped.

3.4 Termination and Prefix-Freeness

Because n is odd, the appended `eof` symbol occupies the even position $n + 1$ and is processed with the final data block in the first pass. If this last pair has zero-based pair index i , the algorithm additionally appends the value 0, regarded as an element of

$$[B - 4i - 4].$$

This artificial value supplies the short range needed to pair with the final long range in the second pass.

During decoding, the inverse local transformations recover enlarged-alphabet symbols in order. The decoder stops immediately upon recovering `eof`, so the artificial zero is not part of the original message. Property 3 of SOLE bounds the required decoding window by a constant number of nearby output blocks. In particular, the window that reveals `eof` lies within the available encoding, so the decoder never has to read past its end. The resulting code is therefore self-delimiting and prefix-free.

3.5 A Concrete Example

Take

$$n = 3, \quad B = 32, \quad (a_1, a_2, a_3) = (5, 7, 31).$$

Here $\text{eof} = 32$, and the augmented sequence is $(5, 7, 31, 32, 0)$. For the first pair, Pass 1 gives

$$7 \cdot 33 + 5 = 236 = 7 \cdot 32 + 12,$$

and hence $(12, 7)$. For the second pair,

$$32 \cdot 33 + 31 = 1087 = 38 \cdot 28 + 23,$$

and hence $(23, 38)$. The intermediate sequence is therefore

$$(12, 7, 23, 38, 0).$$

Pass 2 leaves the first 12 in place and regroups the remaining values. For $(7, 23)$, whose ranges have sizes 36 and 28,

$$23 \cdot 36 + 7 = 835 = 26 \cdot 32 + 3,$$

which produces $(3, 26)$. The final group gives

$$0 \cdot 40 + 38 = 38 = 1 \cdot 32 + 6,$$

which produces $(6, 1)$. Thus the complete output is

$$(12, 3, 26, 6, 1).$$

3.6 Locality and Updates

Property 3 makes the locality quantitative. Output blocks $2i$ and $2i + 1$ are determined by input blocks $2i - 1, \dots, 2i + 2$, while input blocks $2i + 1$ and $2i + 2$ can be recovered from output blocks $2i, \dots, 2i + 3$. Thus a constant number of input blocks determines a constant number of output blocks, and conversely.

Changing one input block consequently requires recomputing only a constant neighborhood of the encoding. Appending a block or truncating the stream can likewise be reduced to a constant number of local writes near the end. This worst-case locality distinguishes SOLE from a global mixed-radix conversion.

3.7 Variants and Practical Remarks

The basic presentation favors a transparent termination argument and uses $n + 2$ output blocks. A tighter termination treatment reduces the boundary waste and yields an $n + 1$ -block version. The paper also gives the symmetric LOSE encoding, which reverses the short/long placement when that orientation is more convenient.

For practical block sizes such as $b = 32$ or $b = 64$, all operands fit in a constant number of machine words, and the quotient–remainder transformations can be implemented with standard word arithmetic. SOLE is therefore not merely an asymptotic counting argument.

3.8 From SOLE to Information Carriers

The non-power-of-two values temporarily left in SOLE’s short and long ranges are instances of a *spill*: information that has not yet been committed to standard binary memory. Later blocks help commit an old spill while leaving a new, smaller spill. Section 4 abstracts this mechanism into the information-carrier lemma, independent of SOLE’s particular alternating ranges.

4 Information Carriers

4.1 The Spill Problem

Suppose an intermediate value satisfies $y \in [Y]$, where $[Y] = \{0, 1, \dots, Y-1\}$. If Y is not a power of two, storing y independently requires $\lceil \log_2 Y \rceil$ bits, although it contains only $\log_2 Y$ bits of information. Repeating this upward rounding creates linear redundancy.

The paper calls such a value a *spill*. A spill is not lost information and does not indicate a failed encoding; it is a reversible intermediate state whose range has not yet been embedded into a standard binary memory region.

4.2 The Information Carrier Abstraction

Let

$$x \in [X], \quad y \in [Y].$$

Here x is an information carrier and y is the old spill. The goal is an injective local transformation

$$(x, y) \mapsto (m, s),$$

where

$$m \in [2^M], \quad s \in [S].$$

The value m is an M -bit memory content, while s is the new spill.

The asymmetry in the decoding requirement is essential: y must be recoverable from m alone, whereas x may be recovered from m and s . Thus the old spill is truly committed to the current memory word. If recovering it also required the new spill, queries could follow an unbounded chain of successor spills and locality would be lost.

4.3 The Information Carrier Lemma

Lemma 1 (Information carrier). *Let $x \in [X]$ and $y \in [Y]$, with the parameters and values fitting in a constant number of Word RAM words. There are integers M and S , and an injective mapping*

$$[X] \times [Y] \longrightarrow [2^M] \times [S],$$

such that y is recoverable from m alone, x is recoverable from (m, s) , and

$$S = O(\sqrt{X}),$$

$$2^M = O(Y\sqrt{X}),$$

and

$$M + \log_2 S - \log_2 X - \log_2 Y = O\left(\frac{1}{\sqrt{X}}\right).$$

The mapping and its inverse take $O(1)$ time.

4.4 Construction

Choose M so that $2^M = \Theta(Y\sqrt{X})$, with constants chosen to make the following integer positive, and define

$$C = \left\lfloor \frac{2^M}{Y} \right\rfloor.$$

Then $C = \Theta(\sqrt{X})$. Split the carrier into a quotient and remainder:

$$c = x \bmod C,$$

$$s = \left\lfloor \frac{x}{C} \right\rfloor,$$

and pack the old spill together with the remainder by setting

$$m = yC + c.$$

The new spill has range size

$$S = \left\lceil \frac{X}{C} \right\rceil.$$

Because $c < C$ and $y < Y$,

$$m < YC \leq 2^M,$$

so m is a valid M -bit memory content.

4.5 Decoding and Correctness

The old spill and the carrier remainder are obtained from m by

$$y = \left\lfloor \frac{m}{C} \right\rfloor,$$

$$c = m \bmod C.$$

In particular, m independently recovers y . Given the new spill as well, the carrier is reconstructed as

$$x = sC + c.$$

These equations uniquely recover both inputs, proving that the mapping is injective and that no information is discarded.

4.6 Redundancy Analysis

The redundancy has two independent rounding sources. First, the floor in the definition of C leaves some of the 2^M memory states unused when $[Y] \times [C]$ is packed into memory. Its contribution is

$$R_1 = M - \log_2(YC) = O\left(\frac{1}{C}\right).$$

Indeed, $2^M < Y(C+1)$, so the ratio between the available and used state counts is less than $1 + 1/C$, whose logarithm is $O(1/C)$.

Second, the ceiling in $S = \lceil X/C \rceil$ may add an incomplete final quotient class when $[X]$ is embedded into $[C] \times [S]$. This contribution is

$$R_2 = \log_2 C + \log_2 S - \log_2 X = O\left(\frac{C}{X}\right).$$

Here $CS < X + C$, so the represented state count exceeds X by a factor smaller than $1 + C/X$.

The total redundancy is therefore

$$R_1 + R_2 = O\left(\frac{1}{C} + \frac{C}{X}\right).$$

Balancing the loss from floor rounding against the loss from ceiling rounding gives

$$C = \Theta(\sqrt{X}),$$

and hence

$$R_1 + R_2 = O\left(\frac{1}{\sqrt{X}}\right).$$

The same choice yields $S = O(\sqrt{X})$ and $2^M = O(Y\sqrt{X})$, completing the claimed bounds.

4.7 Constant-Time Implementation

Encoding and decoding use only a constant number of additions, multiplications, divisions, and modulo operations. The range parameters are fixed for an instance and can be precomputed. Under the paper's Word RAM assumptions, these word-level operations take $O(1)$ time. Division by a fixed constant can also be replaced by multiplication with a precomputed reciprocal and a constant amount of correction when appropriate.

4.8 Why the Lemma Is Powerful

The lemma is not an ordinary compression statement: it specifies which part of the input becomes independently accessible. The old spill is committed locally to m , so decoding does not create an infinite dependence on later carriers. Only the unresolved part of the current carrier continues as the new spill. Moreover, increasing the carrier universe X reduces the per-step redundancy as $O(1/\sqrt{X})$.

This mechanism is reused in the paper's vector representation and in its online prefix-free code. The next application groups vector information into large enough carriers that spills can be absorbed with negligible loss while reads and updates remain local.

5 Optimal-Space Vector Representation

5.1 The Representation Problem

Consider a vector

$$A[1 \dots n], \quad A[i] \in \Sigma,$$

where the alphabet has size Σ . The vector contains $n \log_2 \Sigma$ bits of information, so an injective binary representation needs at least

$$\lceil n \log_2 \Sigma \rceil$$

bits. Treating the entire vector as one integer in $[\Sigma^n]$ attains this bound, but reading or changing one coordinate then requires a global base conversion. At the other extreme, storing every coordinate in $\lceil \log_2 \Sigma \rceil$ bits gives local access but incurs linear redundancy whenever Σ is not a power of two.

The objective is to attain the exact information-theoretic bound while supporting both reads and writes in constant time. The information carrier lemma from Section 4 supplies the required local primitive: it commits an old spill to binary memory with low redundancy while leaving a bounded new spill.

5.2 Grouping Symbols into Carriers

Partition consecutive coordinates into groups and interpret each group as a mixed-radix value

$$x_i \in [X].$$

The paper chooses

$$X = \Theta(n^2).$$

Thus a carrier contains $\Theta(\log_\Sigma n)$ original symbols, and the number of carriers is

$$N = O\left(\frac{n}{\log_\Sigma n}\right).$$

Boundary groups are assigned their deterministically known ranges; this does not change the asymptotic calculation.

This grouping is what amortizes the rounding loss. One application of the information carrier lemma has redundancy

$$O\left(\frac{1}{\sqrt{X}}\right).$$

For $X = \Theta(n^2)$, this is $O(1/n)$ per carrier, and hence

$$N \cdot O\left(\frac{1}{n}\right) = o(1).$$

Rather than rounding each small alphabet symbol separately, the construction performs one rounding operation on a carrier containing logarithmically many symbols, making the sum of all rounding losses vanish asymptotically.

5.3 A Sequential First Attempt

Let the carriers be $x_1, x_2, \dots, x_N$. Starting with a null spill y_0 , represented by a singleton range, step i applies the information carrier mapping

$$(x_i, y_{i-1}) \mapsto (m_i, y_i).$$

Here m_i is written to an M_i -bit memory field, and $y_i \in [Y_i]$ is passed to the next step. The lemma ensures $Y_i = O(\sqrt{X})$, even though its exact value may depend irregularly on Y_{i-1} . After the last step, the final spill y_N is stored separately.

5.4 Redundancy of the Sequential Construction

The difference between the stored length and the information in the carriers is

$$\lceil \log_2 Y_N \rceil + \sum_{i=1}^N M_i - N \log_2 X.$$

Using the chain rule for redundancy, this is at most

$$1 + \sum_{i=1}^N (M_i + \log_2 Y_i - \log_2 Y_{i-1} - \log_2 X).$$

Each parenthesized term is precisely the redundancy of one information carrier transformation. The positive $\log_2 Y_i$ term at one step cancels the negative term at the next, while the leading one accounts for rounding up the space used by the final spill. Consequently the total redundancy is

$$1 + N \cdot O\left(\frac{1}{\sqrt{X}}\right) = 1 + o(1).$$

Crucially, intermediate spills are passed forward rather than independently stored in rounded-up binary fields. They therefore do not contribute one rounding bit per step.

5.5 Constant-Time Reads and Writes

Suppose a requested coordinate lies in x_j . If $j < N$, the asymmetric decoding guarantee of the lemma recovers the old spill y_j from m_{j+1} alone. The pair (m_j, y_j) then recovers x_j , from which the coordinate is extracted. For $j = N$, the separately stored final spill y_N replaces the nonexistent m_{N+1} . Thus decoding never begins at y_0 and touches only a constant number of memory fields.

To change x_j , first re-encode step j . Its new output spill y_j may differ, so step $j + 1$ is also re-encoded when $j < N$. In the carrier mapping of Section 4, the new spill is the quotient part of the carrier and is independent of the old spill. Since x_{j+1} has not changed, this second encoding can retain the same y_{j+1} ; only its memory content changes. Propagation therefore stops after one neighboring step. When $j = N$, only m_N and the separately stored y_N are changed. Both reads and writes take $O(1)$ time.

5.6 Why the Sequential Construction Is Insufficient

The sequential construction is correct, space-efficient, and locally accessible. Its defect is instead the amount of nonuniform precomputation. The spill universes Y_i may all differ, so different steps may require different values of M_i , spill-size parameters, information carrier constants, memory offsets, and fixed division or multiplication constants. It may therefore require

$$N = O\left(\frac{n}{\log n}\right)$$

sets of precomputed parameters, rather than the $O(\log n)$ word constants required by the theorem.

5.7 The Implicit Binary-Tree Representation

The final construction places the carriers in an implicit binary tree using heap numbering:

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1, \quad \text{parent}(i) = \lfloor i/2 \rfloor.$$

No pointers are stored. A node combines the spills received from its two children into a mixed-radix value y_i , and uses its own carrier in the mapping

$$(x_i, y_i) \mapsto (m_i, s_i).$$

The memory content m_i is written in the node's assigned memory area, and the new spill s_i is passed to its parent. A missing child contributes the unique value in a singleton, deterministically known range; leaves therefore have a fixed null input range. This handles incomplete boundary subtrees without pointers or an auxiliary data structure. The root spill s_1 is stored separately.

5.8 Queries and Updates in the Tree

For a nonroot node v , the parent's memory content independently reveals the combined old spill at the parent. Its mixed-radix components include the spill s_v sent by v . Combining s_v with m_v recovers x_v , and the desired coordinate is then extracted. For the root, the separately stored s_1 and m_1 recover x_1 . Although spills move upward while the representation is encoded, a query never follows a root-to-node path and does not recurse; it examines only the node and its parent.

An update re-encodes the target node. Its outgoing spill may change, so the parent is re-encoded as well. The parent's carrier remains fixed, and the information carrier mapping consequently retains the parent's outgoing spill while absorbing the changed child spill into new memory content. Propagation stops there. Updating the root changes only its memory content and separately stored spill. Hence both operations take $O(1)$ time.

5.9 Reducing the Number of Precomputed Constants

Parameter sharing is the reason for using the tree. At a fixed level, for an appropriate height k , node subtrees occur in only three classes:

- (1) a prefix of nodes has complete binary subtrees of height k ;
- (2) at most one node has an incomplete subtree;
- (3) the remaining nodes have complete binary subtrees of height $k - 1$.

Nodes in the same class and level have the same subtree size. By induction, their two child spill ranges are identical, so they share the information carrier parameters, memory field sizes, and the rule for locating those fields. It suffices to store, for each class, its size, the first memory location, both child spill sizes, and the constants used by the lemma.

There are only constantly many classes per level, and the tree height is

$$O(\log N) = O(\log n).$$

The total number of precomputed word constants is therefore $O(\log n)$. For heap index i , its level is

$$\lfloor \log_2 i \rfloor,$$

which is computable by a constant number of Word RAM operations. The level then identifies the node's class and the shared parameters and memory locations in constant time.

5.10 The Final-Bit Issue

The direct construction yields

$$\lceil n \log_2 \Sigma + o(1) \rceil$$

bits, which can in principle equal $\lceil n \log_2 \Sigma \rceil + 1$. The paper removes this possible last bit by increasing the block, and hence carrier, universe. Arithmetic then uses $O(c \log n)$ -bit integers for a fixed constant c , still a constant number of Word RAM words, and the accumulated redundancy becomes $O(n^{-c})$.

For fixed Σ , the paper invokes a lower bound from the theory of linear forms in logarithms: unless $n \log_2 \Sigma$ is already integral, its distance from an integer is at least inverse polynomial in n . Choosing c sufficiently large makes the remaining redundancy smaller than this separation, ensuring

$$n \log_2 \Sigma + O(n^{-c}) \leq \lceil n \log_2 \Sigma \rceil.$$

Only this separation consequence is needed; the underlying number-theoretic theorem is not reproved here.

5.11 The Vector Representation Theorem

Theorem 1. *On the Word RAM, a vector $A[1 \dots n]$ over an alphabet of size Σ can be represented using*

$$\lceil n \log_2 \Sigma \rceil$$

bits while supporting reads and writes in $O(1)$ time. The representation requires $O(\log n)$ precomputed word constants.

Proof. Grouping symbols into carriers of universe $X = \Theta(n^2)$ reduces the sum of local rounding redundancies to $o(1)$. The tree preserves the lemma's local recovery asymmetry, so a carrier is read from its node and parent and an update is absorbed after re-encoding at most those two nodes. Nodes of the same level and subtree class share parameters, leaving only $O(\log n)$ precomputed word constants. Finally, enlarging the carriers reduces the residual error below the inverse-polynomial separation from the next integer, which gives the exact $\lceil n \log_2 \Sigma \rceil$ -bit bound. $\square$

6 Online Prefix-Free Encoding

6.1 The Online Universal-Coding Problem

The paper's second main result concerns a binary stream whose final length is not known when encoding begins. A valid encoding must be prefix-free, so the decoder can identify

the message boundary without receiving the length separately. An offline Elias code can place an already known length before the payload, but an online encoder cannot emit such a header before learning the end of the stream. Adding an integral end marker to every fixed-size block is online, but incurs linear redundancy.

The construction combines slowly growing blocks, an early termination symbol, and the local mapping of Section 4. It establishes a universal code of length

$$n + \log_2 n + O(\log \log n)$$

for an n -bit input, using $O(\log n)$ bits of working memory and constant time per word-level block operation. This develops the simplified fixed-block intuition of SOLE from Section 3 into a code for an unknown-length binary stream.

6.2 Slowly Growing Blocks

Block sizes are determined online from the amount of input already seen. When the observed prefix has the current scale denoted by n in the paper's local description, the next complete block has

$$2k_i = 2 \lceil \log_2 n \rceil$$

bits. This use of n does not assume knowledge of the eventual stream length. The complete block consists of two k_i -bit components and is interpreted as

$$x_i \in [2^{k_i}]^2.$$

Thus k_i is the component length, whereas the full block length is $2k_i$.

The universe of a complete block has size 2^{2k_i} . Since the information-carrier loss on a universe of size X is $O(1/\sqrt{X})$, this choice gives a local loss of

$$O\left(\frac{1}{\sqrt{2^{2k_i}}}\right) = O(2^{-k_i}).$$

As the stream grows, later rounding losses consequently become smaller.

6.3 Handling the Final Partial Block

Let N be the final number of blocks. The first $N - 1$ blocks are complete, whereas block N may contain between one and $2k_N$ bits. The main construction encodes the complete prefix, after which it appends an Elias encoding for the last partial block. This special encoding supplies only the boundary information for a block of logarithmic size; it does not place the unknown total length before the entire stream. Its overhead is

$$O(\log k_N) = O(\log \log n).$$

6.4 Pass One: Embedding an Early EOF Symbol

Pass one enlarges the alphabet of each complete block from $[2^{k_i}]^2$ to

$$[2^{k_i}] \times ([2^{k_i}] \cup \{\text{eof}\}).$$

If block $N - 2$ originally equals

$$(a, b) \in [2^{k_{N-2}}]^2,$$

the encoder replaces it by (a, eof) . The displaced component b is not discarded: it is appended in plaintext as part of the special tail.

The encoder maintains a three-block lag using an $O(\log n)$ -bit buffer. When a block first arrives, the encoder cannot know whether it will eventually be block $N - 2$. It therefore delays the last constant number of blocks and chooses the termination position only after the input ends. No previously emitted output is rewritten.

The position $N - 2$, rather than the last block, is forced by decoding locality. Recovering an input block requires information committed to the next output block. Moving termination sufficiently far forward ensures that the decoder learns it before exhausting the ordinary encoded stream.

6.5 Redundancy of Alphabet Enlargement

The original universe of block i has 2^{2k_i} states, while the enlarged universe has $2^{k_i}(2^{k_i} + 1)$ states. Its information cost is

$$\begin{aligned} \log_2(2^{k_i}(2^{k_i} + 1)) - 2k_i \\ = \log_2(1 + 2^{-k_i}) = O(2^{-k_i}). \end{aligned}$$

The sum is analyzed by block scale. For a fixed K , the slowly growing partition contains

$$O\left(\frac{2^K}{K}\right)$$

blocks satisfying $k_i = K$. Since each contributes $O(2^{-K})$, the entire scale contributes

$$O\left(\frac{2^K}{K}\right) O(2^{-K}) = O\left(\frac{1}{K}\right).$$

Summing this harmonic bound over $1 \leq K \leq \log_2 n$ gives

$$\sum_{K=1}^{\log_2 n} O\left(\frac{1}{K}\right) = O(\log \log n).$$

6.6 Pass Two: Conversion through Information Carriers

Pass two converts the enlarged alphabets to binary. As in the sequential carrier chain preceding the tree construction of Section 5, it applies the information-carrier mapping in order:

$$(x_i, s_{i-1}) \mapsto (m_i, s_i).$$

Here x_i is the current enlarged-alphabet block, s_{i-1} is the old spill, m_i is binary memory content that can be emitted, and s_i is the new spill passed to the next block. The asymmetric inverse guaranteed in Section 4 commits the old spill to m_i , which is precisely the locality needed by the decoder.

The carrier domain at step i has size $O(2^{2k_i})$, so the lemma adds $O(2^{-k_i})$ redundancy. The same scale decomposition used above bounds the cumulative carrier redundancy by $O(\log \log n)$. The proof of the lemma need not be repeated: its bounded spill and constant number of word arithmetic operations apply directly to these growing domains.

6.7 Why the Decoder Stops in Time

The termination symbol occurs in input block $N - 2$. After reading output block $N - 1$, the local inverse mapping has recovered the spill needed to decode input block $N - 2$, and hence reveals its second component eof. The decoder therefore sees termination before it would need to read beyond the ordinary encoded stream.

Upon detecting eof, the decoder stops the sequential carrier procedure and switches to the special tail. It decodes the final partial block and reads the displaced component b , restores b as the second half of block $N - 2$, and outputs the remaining original bits. Because the switch occurs immediately after output block $N - 1$, the special tail is not mistaken for additional ordinary carrier blocks.

6.8 Total Encoding Length

Replacing the second component of block $N - 2$ by termination and moving that component to the tail costs at most $k_{N-2} \leq \log_2 n$ bits. This is one termination mechanism, not two independent charges for the marker and the displaced data. Alphabet enlargement and the sequential carrier mappings each add $O(\log \log n)$, and the final partial block adds another $O(\log \log n)$. Consequently,

$$\begin{aligned} \text{encoding length} &= n + \underbrace{\log_2 n}_{\text{early termination}} + \underbrace{O(\log \log n)}_{\text{alphabet enlargement}} \\ &\quad + \underbrace{O(\log \log n)}_{\text{carrier mappings}} + \underbrace{O(\log \log n)}_{\text{partial block}} \\ &= n + \log_2 n + O(\log \log n). \end{aligned}$$

6.9 Memory and Running Time

The encoder and decoder retain a constant number of $O(\log n)$ -bit blocks, one bounded spill, and a constant number of counters and block parameters. Their working memory is therefore $O(\log n)$ bits.

Each pass performs a constant number of additions, multiplications, divisions, and quotient-remainder operations per block. Under the paper's Word RAM assumptions this is $O(1)$ time per block, or per word-level operation. The special ending processes only a constant number of blocks; the total processing time over a stream remains linear in the number of processed blocks, rather than constant for the entire input.

6.10 Significance of the Construction

SOLE demonstrates with fixed blocks how short and long ranges can carry a non-binary spill. The general carrier lemma removes that simplifying structure and supports the varying alphabets required here. Slowly growing blocks make local rounding losses summable, while early termination aligns the code boundary with the lemma's local inverse. The same primitive that gave exact-space vector representation thus also yields the paper's second main result: a low-memory online code whose redundancy is within $O(\log \log n)$ of the Elias-scale bound.

7 Related Work

The paper lies at the intersection of two subjects: succinct representation with local access and online self-delimiting coding. Its common technical primitive is a local, reversible conversion between a finite nonbinary alphabet and binary memory. The two applications should nevertheless be compared with different literatures: redundancy and query lower bounds for the vector result, and universal coding and cryptographic domain extension for the online result [6].

7.1 Succinct Data Structures and Redundancy

Since Jacobson’s foundational work, succinct data structures have sought to store an object near its information-theoretic minimum while retaining useful operations [12]. A vector in Σ^n has Σ^n possible values and therefore needs at least $\lceil n \log_2 \Sigma \rceil$ bits. Encoding coordinates independently uses $n \lceil \log_2 \Sigma \rceil$ bits and wastes

$$n(\lceil \log_2 \Sigma \rceil - \log_2 \Sigma)$$

bits. Treating the vector as one integer in $[\Sigma^n]$ eliminates that rounding loss but makes a coordinate operation a global base conversion.

Earlier succinct-index techniques reduced the redundancy while retaining fast access. Golynski et al. obtained constant access with $O(n/\log^2 n)$ redundancy in the relevant regime, and Pătraşcu’s recursive “Succincter” framework gave a family of time–redundancy tradeoffs [11, 15]. The result reviewed here is sharper for the abstract fixed-alphabet vector: on its Word RAM, it occupies exactly $\lceil n \log_2 \Sigma \rceil$ bits and supports both reads and writes in constant time. This is a theorem about this vector problem, not a claim that all succinct structures can eliminate redundancy.

7.2 Rank, Select, and Lower Bounds

Rank and select are central navigation primitives in many succinct structures. Their space–time limitations motivated general lower-bound programs in the cell-probe and bit-probe models [8, 10, 16, 17]. These bounds concern specified query problems, encodings, and computational models; they cannot be transferred to every finite-alphabet representation merely because the latter is succinct.

The vector theorem does not contradict the rank/select lower bounds. It shows instead that plain coordinate access admits a representation that does not implement rank/select and therefore avoids the machinery to which those bounds apply. Conversely, it does not supply rank, select, or general set membership. The paper itself identifies dictionaries as a harder target and presents local base conversion as evidence that alternative technical routes can sometimes bypass a familiar barrier, not as a refutation of the barrier for the problems where it was proved.

7.3 Arithmetic Coding and Locality

Arithmetic coding treats a much broader source model. For independently drawn symbols from a distribution D , it approaches the entropy $nH(D)$, and standard bounded-precision implementations encode and decode streams efficiently [18, 14]. In the uniform case this

objective resembles conversion from base Σ to binary, but the operational guarantees differ.

Renormalization may leave an interval straddling one half. Standard implementations count such outstanding bits until a later symbol resolves their values, at which point a burst of output is emitted. Thus a particular encoded position may depend on many surrounding symbols. This is compatible with good sequential compression, but it does not provide the worst-case local random access, update, and per-block output behavior required here. The comparison is consequently not that arithmetic coding is inferior: it handles nonuniform distributions that this paper does not, whereas the paper specializes to uniform finite alphabets in exchange for strong locality.

7.4 Universal and Prefix-Free Coding

Elias codes are a classical offline method for self-delimiting integer representations [7]. Once the payload length is known, it can be placed before the payload using delta, omega, or related iterated descriptions. Elias-type offline codes achieve a sharper iterated-log expression than merely $n + \log_2 n + O(\log \log n)$.

The online problem is different because the encoder must begin before the final length is known. Buffering a fixed block and adding an integral EOF flag per block is online but has linear redundancy. Maurer and Sjödin’s work on single-key message authentication supplied the cryptographic context and conjectured that low-memory online prefix-free encoding required such a linear cost [13]. The reviewed paper refutes that conjecture by obtaining

$$n + \log_2 n + O(\log \log n)$$

bits, within $O(\log \log n)$ of the Elias-scale optimum, together with $O(\log n)$ working memory and constant time per word-level operation. Its novelty is the simultaneous online, locality, memory, and redundancy guarantee, not a replacement for every offline Elias code.

7.5 Cryptographic Domain Extension

Many domain-extension constructions iterate a fixed-input-length primitive over message blocks. CBC-MAC, cascade PRFs (including the GGM viewpoint), and Merkle–Damgård-style hashing are prominent examples [3, 2, 9, 5]. If one valid encoded message is a prefix of another, the shorter computation is an ancestor state of the longer computation. This relation enables the corresponding extension attacks: an adversary may continue processing from a state associated with the shorter message.

Prefix-free input encoding is a generic way to exclude that ancestor–descendant relation. Other constructions use additional keys, outer applications, strengthened finalization, or dedicated domain-extension transforms; NMAC/HMAC and multi-property-preserving transforms illustrate this design space [1, 4]. The paper did not invent CBC-MAC, HMAC, or Merkle–Damgård. Appendix A uses them as motivation: lowering the redundancy and state cost of online prefix-free encoding makes “PFE then cascade” a cleaner generic option in settings whose security proof accepts prefix-free domains. This observation does not remove the need to analyze the concrete primitive, padding rule, and security model.

7.6 Subsequent Developments after 2010

Later work continued to reduce redundancy in succinct structures, but the relationship must be stated carefully. Yu’s 2019 rank structure matches the Pătraşcu–Viola cell-probe tradeoff over a broad parameter range using approximate nonnegative tensor decomposition [19]. It concerns rank rather than plain coordinate access and is evidence of progress on the barrier discussed in 2010; it is not presented as a direct extension of the local base-conversion construction.

Yu’s static Las Vegas dictionary develops a toolkit of fractional-length strings from spillover representations, with local concatenation and fusion, and uses it in a near-optimal dictionary and zero-order sequence compression [20]. This is substantively related to the principle of accumulating fractional information before rounding. Its stated lineage is the spillover representation in “Succincter,” however, so similarity to the information-carrier viewpoint does not by itself establish direct descent from the 2010 paper. These examples show that spillover and fractional-bit methods remained useful, while also showing why citation alone is insufficient evidence of a technical inheritance claim.

The literature checked for this review did not establish an equally clear post-2010 line that directly improves the paper’s online prefix-free theorem under the same online-memory and worst-case-locality requirements. We therefore avoid claiming either that the theorem became a standard practical method or that later universal and locally decodable compression results are direct continuations without a model-by-model proof.

7.7 Positioning the Paper

The paper is best understood as a local, reversible, low-redundancy base conversion result for uniform finite alphabets. It is not a general-purpose compressor, a rank/select data structure, or a new cryptographic primitive. Its conceptual strength is that one information-carrier mechanism yields two different consequences: exact-space mutable vectors and a low-memory online prefix-free code close to the offline universal-coding scale.

The contribution is strongest as a theoretical separation between integral rounding and inherent information cost. Practical applicability is more conditional: it depends on Word RAM arithmetic, precomputed constants, block parameters, and the surrounding security or storage system. Extending the same locality guarantees to biased distributions, or transferring the method to richer query problems, remains outside the proved scope.

8 Correctness and Complexity Analysis

8.1 Proof Architecture

The preceding constructions form one proof chain rather than four independent encodings. SOLE, in Section 3, exhibits a reversible local change of finite radix. Lemma 1 isolates the invariant behind that example: an old spill is committed to an integer number of memory bits, while a bounded new spill carries only the unresolved part forward. The vector result organizes this invariant in an implicit tree, and the online prefix-free result organizes it in a stream with delayed termination [6]. This section audits the proof obligations that are shared across those applications and records the computational assumptions on which their time bounds depend.

8.2 Correctness of the SOLE Transform

Let $B = 2^b$, let $[B] = \{0, \dots, B-1\}$, and identify `eof` with B , so an enlarged-alphabet symbol belongs to $[B+1]$. In Pass 1, the pair $(x, y) \in [B+1]^2$ is combined as

$$z = y(B+1) + x$$

and divided by $B-4i$:

$$a = z \bmod (B-4i), \quad q = \left\lfloor \frac{z}{B-4i} \right\rfloor.$$

The remainder automatically satisfies $a \in [B-4i]$. The range of the quotient follows from the full capacity calculation

$$\begin{aligned} (B-4i)(B+4i+4) - (B+1)^2 \\ = 2B - 16i(i+1) - 1. \end{aligned}$$

Thus

$$(B+1)^2 \leq (B-4i)(B+4i+4)$$

is equivalent to

$$B \geq 2(2i+1)^2 - \frac{3}{2}.$$

Basic SOLE assumes $B \geq 2n^2$, and every processed pair satisfies $2i+1 \leq n$, so the required inequality holds. Since $z < (B+1)^2$, it follows that $q \in [B+4i+4]$.

The inverse is explicit. From (a, q) , it reconstructs

$$z = q(B-4i) + a,$$

followed by

$$x = z \bmod (B+1), \quad y = \left\lfloor \frac{z}{B+1} \right\rfloor.$$

Both uniqueness claims are instances of Euclidean division. After Pass 1, Pass 2 regroupes a long and a short value. For $i \geq 1$, the pair belongs to $[B+4i] \times [B-4i]$, and

$$(B+4i)(B-4i) = B^2 - 16i^2 < B^2.$$

It therefore has a unique representation as two base- B digits. The first $[B]$ value is not part of this shifted pairing, so the strict inequality is used only where $i \geq 1$.

For the basic odd-length presentation, write $n = 2r+1$. The appended `eof` is at position $2r+2$, and the algorithm appends

$$0 \in [B-4r-4]$$

after Pass 1 to complete the final long-short pair. For even n , the basic presentation pads to the odd case and can use one more boundary block; the tighter variant in the paper handles both parities with an early EOF mechanism. In either case, termination needs more than the existence of a distinguished symbol. SOLE's Property 3 states that input blocks $2i+1$ and $2i+2$ are decoded from output blocks $2i, \dots, 2i+3$. The window that reveals the final EOF lies inside the produced encoding, so the decoder stops without probing beyond the file. The same property shows the exact locality guarantee: encoding and decoding use four-block neighborhoods, not a single block.

8.3 Correctness and Redundancy of the Information-Carrier Lemma

Section 4 gives the construction; here we make its proof obligations explicit. For $x \in [X]$, $y \in [Y]$, choose M so that

$$C = \left\lfloor \frac{2^M}{Y} \right\rfloor = \Theta(\sqrt{X}), \quad S = \left\lceil \frac{X}{C} \right\rceil,$$

with $C \geq 1$. Define

$$c = x \bmod C, \quad s = \left\lfloor \frac{x}{C} \right\rfloor, \quad m = yC + c.$$

Because $0 \leq c < C$ and $0 \leq y < Y$,

$$0 \leq m < YC \leq 2^M,$$

so m is a legal M -bit memory value. Euclidean division gives

$$y = \left\lfloor \frac{m}{C} \right\rfloor, \quad c = m \bmod C, \quad x = sC + c.$$

Consequently the map is injective. More importantly, the old spill y is recovered from m alone; recovering it does not initiate a chain through later spills. This asymmetric local recovery condition is what supports the constant-time accesses in both applications.

The two rounding losses can be bounded without suppressing the floor and ceiling. From the definition of C ,

$$YC \leq 2^M < Y(C + 1),$$

and hence

$$\begin{aligned} R_1 &= M - \log_2(YC) \\ &= \log_2 \frac{2^M}{YC} < \log_2 \left(1 + \frac{1}{C} \right) = O\left(\frac{1}{C}\right). \end{aligned}$$

Similarly, $S = \lceil X/C \rceil$ implies $CS < X + C$, so

$$\begin{aligned} R_2 &= \log_2 C + \log_2 S - \log_2 X \\ &= \log_2 \frac{CS}{X} < \log_2 \left(1 + \frac{C}{X} \right) = O\left(\frac{C}{X}\right). \end{aligned}$$

Balancing the two terms at $C = \Theta(\sqrt{X})$ yields

$$R_1 + R_2 = O\left(\frac{1}{\sqrt{X}}\right), \quad S = O(\sqrt{X}), \quad 2^M = O(Y\sqrt{X}).$$

The encoder and decoder use a constant number of word additions, multiplications, divisions, and remainder operations once M, C, S are precomputed.

8.4 Proof of the Optimal-Space Vector Result

In Section 5, Σ denotes the alphabet cardinality. Consecutive coordinates are grouped into carriers from a universe

$$X = \Theta(n^2).$$

For a fixed finite alphabet of size $\Sigma \geq 2$, each carrier contains $\Theta(\log_\Sigma n)$ coordinates and the number of carriers is

$$N = O\left(\frac{n}{\log_\Sigma n}\right).$$

Lemma 1 contributes $O(1/n)$ redundancy per carrier, so the total carrier redundancy is

$$N O\left(\frac{1}{n}\right) = O\left(\frac{1}{\log_\Sigma n}\right) = o(1).$$

The sequential first attempt already has this space bound and local access. Its failure is nonuniform precomputation: irregular spill universes can require $O(N)$ distinct parameter sets and memory offsets. The implicit binary tree reduces that cost. On any level its nodes form at most three classes: complete subtrees of height k , at most one incomplete subtree, and complete subtrees of height $k - 1$. Nodes in one class receive identical child-spill universe sizes and therefore share carrier parameters. There are only constantly many classes per level and $O(\log n)$ levels, giving the $O(\log n)$ precomputed word constants in Theorem 1.

The tree does not cause logarithmic query time. To recover the carrier at a nonroot node v , the algorithm uses the parent's memory to recover the spill sent by v , then combines that spill with v 's own memory. It touches the node and its parent, not a root-to-leaf path. The root uses its separately stored spill. A coordinate is then extracted from the recovered carrier by fixed-radix word arithmetic.

An update is equally local. Re-encoding v may change its outgoing spill, so its parent must also be re-encoded. In the carrier construction, however, the new spill is the carrier quotient

$$s = \left\lfloor \frac{x}{C} \right\rfloor,$$

which does not depend on the old-spill value. The parent's own carrier and parameters are unchanged; therefore its outgoing spill is unchanged even though its memory content absorbs the modified child spill. No update reaches the grandparent. Roots, leaves, and the unique incomplete subtree use the same argument with separately stored or singleton missing-child spills.

Finally, $n \log_2 \Sigma + o(1)$ is not by itself the claimed exact integer bound. Storing the final root spill gives initially

$$\lceil n \log_2 \Sigma + o(1) \rceil,$$

which could be one bit larger than $\lceil n \log_2 \Sigma \rceil$. The paper enlarges the carriers so that the residual redundancy is $O(n^{-c})$ for any fixed c , while arithmetic still occupies a constant number of $O(\log n)$ -bit words. For fixed Σ that is not a power of two, a linear-forms-in-logarithms bound gives an inverse-polynomial lower bound on the nonzero distance from $n \log_2 \Sigma$ to an integer. Choosing c large enough keeps the residual below the next-integer gap, so the final ceiling absorbs it. Power-of-two alphabets have an integral information bound and admit direct fixed-width packing. The number-theoretic theorem is used only for this last rounding step, not for locality or the carrier lemma.

8.5 Proof of the Online Prefix-Free Result

To avoid overloading n , let t be the number of bits observed when the next block size is chosen, let n be the final input length, and let N be the final number of blocks. A complete block has two halves of length

$$k_i = \lceil \log_2 t \rceil$$

and hence universe $X_i = 2^{2k_i} = \Theta(t^2)$, up to scale constants. The carrier loss is therefore $O(2^{-k_i})$. Enlarging one half-alphabet from $[2^{k_i}]$ to $[2^{k_i}] \cup \{\text{eof}\}$ costs

$$\begin{aligned} \log_2(2^{k_i}(2^{k_i} + 1)) - 2k_i \\ = \log_2(1 + 2^{-k_i}) = O(2^{-k_i}). \end{aligned}$$

At scale $k_i = K$, there are $O(2^K/K)$ blocks because the scale contains $O(2^K)$ input bits and each block has $\Theta(K)$ bits. Each of the two losses thus contributes $O(1/K)$ at that scale. Since the largest scale is $O(\log n)$,

$$\sum_{K=1}^{\lceil \log_2 n \rceil} O\left(\frac{1}{K}\right) = O(\log \log n).$$

Prefix-freeness depends on the timing of EOF discovery. A three-block lag keeps a constant number of blocks in an $O(\log n)$ -bit buffer. If complete input block $N - 2$ was (a, b) , the encoder replaces it by (a, eof) and saves the displaced half b in the special tail. The last incomplete block N is also encoded in that tail with an Elias code. No already emitted output is rewritten.

After reading ordinary output block $N - 1$, the sequential carrier inverse has the spill required to reconstruct input block $N - 2$. It therefore sees **eof** before requesting another nonexistent ordinary output block, stops the carrier decoder, and switches to the self-delimiting tail. The tail restores b and the final partial block. The length decomposition is

$$\begin{aligned} \text{length} &= \underbrace{n}_{\text{payload}} + \underbrace{\log_2 n}_{\text{displaced half / early EOF}} \\ &+ \underbrace{O(\log \log n)}_{\text{alphabet enlargement}} + \underbrace{O(\log \log n)}_{\text{carrier rounding}} \\ &+ \underbrace{O(\log \log n)}_{\text{incomplete-block tail}} \\ &= n + \log_2 n + O(\log \log n). \end{aligned}$$

The displaced half and the early EOF are one mechanism and are not charged as two separate $\log n$ terms. This is near-optimal, within $O(\log \log n)$ of the Elias-scale bound; it is not an exact reproduction of the full Elias omega iterated-log expression.

The streaming state consists of a current block, a bounded spill, block-size counters, the constant-lag buffer, and termination metadata. These are a constant number of $O(\log n)$ -bit objects, so the working space is $O(\log n)$ bits. The time guarantee is $O(1)$ per word-level or block operation, not per individual bit.

8.6 Computational Model and Complexity Assumptions

All constant-time claims use the paper’s Word RAM model. Memory supports random access and is divided into w -bit words with $w = \Omega(\log n)$, so indices and the ordinary carrier parameters fit in a word. Addition, multiplication, and division are unit-cost operations; remainder is obtained with quotient–remainder arithmetic. Each local transformation uses a constant number of words, and its universe-dependent constants are precomputed. The vector data structure counts $O(\log n)$ such constants explicitly. The exact-space refinement uses $O(c \log n)$ -bit integers for fixed c , still a constant number of words in the asymptotic model.

These are theoretical model guarantees. They do not imply that hardware division, cache behavior, or a particular implementation has been benchmarked to be faster on every real platform. Likewise, the online bound is stated per block or word operation; translating it to a per-bit engineering cost depends on the chosen block schedule and machine word size.

8.7 Scope of the Guarantees

The paper proves an exact-space representation for the stated uniform finite-alphabet vector, constant-time coordinate reads and writes on its Word RAM, a low-memory online prefix-free code, and local reversible base conversion. It does not prove a locally decodable arithmetic code for biased distributions, a zero-redundancy rank/select structure, or a general dictionary solution. Nor does prefix-free preprocessing automatically prove the security of every cryptographic construction: the primitive, padding, and security model still require a separate argument, as discussed in Section 7. The results also do not assert optimality for arbitrary compression models or superior performance on every physical machine.

9 Strengths, Limitations, and Open Questions

9.1 Strengths

The paper’s main strengths are conceptual rather than incremental: it identifies a specific source of waste in binary representations and shows that it can be eliminated locally, not merely amortized globally.

- (1) **The right enemy.** The paper identifies integral-bit rounding—not entropy, distributional mismatch, or query complexity—as the independent source of the apparent trade-off between optimal space and local access. This diagnosis changes the question from “can we beat information theory” (which is impossible) to “can we reorganise information so that rounding loss does not accumulate” (which the paper answers affirmatively).
- (2) **Local, reversible base conversion.** Rather than encoding the whole object as a single integer or rounding every symbol individually, the paper develops local arithmetic transformations that move information between nearby components. The construction is explicit, computationally lightweight, and provably reversible without loss.

- (3) **Asymmetric decoding as a locality guarantee.** The information-carrier lemma is not merely injective: it stipulates that the old spill must be recoverable from the current memory content alone. This single asymmetry is the syntactic condition that prevents an unbounded dependence chain through successive carriers and is what ultimately delivers $O(1)$ queries and updates in the vector representation and timely EOF discovery in the online code.
- (4) **One primitive, two applications.** The same information-carrier mechanism supports two results that the literature had treated separately: an exact-space mutable vector with random access, and a low-memory online prefix-free code. The difference is only in how carriers are organized—an implicit binary tree for the vector, a sequential chain for the stream.
- (5) **Simultaneous optimality for the vector.** The vector representation achieves the exact information-theoretic bound $\lceil n \log_2 \Sigma \rceil$, supports both reads and writes (not merely queries), and uses $O(1)$ time per operation. Earlier succinct-index techniques reduced redundancy to low-order terms; this paper eliminates it entirely for the fixed-alphabet vector problem.
- (6) **Near-optimal online code with small memory.** The online prefix-free code has length $n + \log_2 n + O(\log \log n)$ while using only $O(\log n)$ bits of working memory and constant time per word-level operation. It refutes the earlier conjecture that small-space online prefix-free encoding necessarily incurs linear redundancy.
- (7) **Explicit construction and analysis.** Every step is constructive: the encoder and decoder are described as concrete arithmetic procedures, the redundancy bounds are derived from explicit floor and ceiling inequalities, and the constant factors are in principle extractable.
- (8) **Conceptually simple, technically non-obvious.** The basic idea—use a larger memory word to absorb the fractional part of a non-power-of-two range—is simple enough to explain in a paragraph. The non-obvious part is that it works: the per-step redundancy can be made $O(1/\sqrt{X})$, vanishingly small for large enough carriers, while the local inverse is preserved by the asymmetric decoding condition.

9.2 Limitations

The paper’s results are subject to several modeling assumptions, scope boundaries, and pragmatic constraints that should be noted.

- (1) **Word RAM dependence.** All $O(1)$ -time guarantees assume the Word RAM model with $w = \Omega(\log n)$. Addition, multiplication, and division are unit-cost operations. On a concrete machine with fixed-width words and non-constant division latency, the practical constant factors differ from the asymptotic guarantee.
- (2) **Multiplication and division as $O(1)$.** The local base-conversion steps repeatedly compute quotients and remainders, which are treated as constant-time operations. In implementations where division is significantly slower than other operations, the constant in the $O(1)$ bound may be large.

- (3) **Uniform finite alphabet.** Both main results assume a uniform distribution over a fixed finite alphabet. The paper does not address biased or unknown distributions, adaptive alphabets, or variable-length symbol sets.
- (4) **Not a general arithmetic coder.** Arithmetic coding handles non-uniform distributions and approaches $nH(D) + O(1)$ bits in expectation, whereas this paper handles the uniform worst case with strong locality. The two techniques address different problems and their guarantees are not comparable in a simple way.
- (5) **No distributional adaptation.** The paper does not discuss how to adapt its base conversion to biased distributions, nor does it provide bounds that degrade gracefully with distributional skew.
- (6) **Not a rank/select or dictionary solution.** The vector representation supports coordinate access, not rank, select, predecessor, or membership queries. It therefore does not directly compete with data structures optimized for those richer query sets, nor does it bypass the lower bounds that apply to them.
- (7) **No hardware benchmarks.** The paper proves asymptotic bounds under the Word RAM model. It does not report implementation benchmarks, cache-performance measurements, or comparisons with practical alternatives on real hardware.
- (8) **Exact-space step uses number theory.** The final rounding argument that absorbs the $o(1)$ residual into the ceiling invokes a lower bound from the theory of linear forms in logarithms. This step is not self-contained within the combinatorial construction and relies on a deep, non-trivial number-theoretic result.
- (9) **Prefix-freeness is not a security proof.** The online code is a prefix-free encoding, not a cryptographic primitive. Using it in cascade constructions still requires a security reduction that accounts for the concrete primitive, padding rule, and adversarial model. The paper itself makes this distinction clear in Appendix A.
- (10) **Limited adoption in practice.** The authors present the construction as a theoretical contribution. There is no evidence in the paper or in the subsequent literature reviewed here that this specific encoding has become a standard in deployed systems, cryptographic libraries, or compression tools.

9.3 Open Questions

Several directions suggested by the paper remain open or have only been partially addressed.

- (1) **Biased distributions.** Can the information-carrier mechanism be adapted to distributions that are not uniform? A natural goal would be a locally decodable arithmetic code that approaches $nH(D)$ while retaining constant-time access to individual symbols.
- (2) **Locally decodable arithmetic coding.** Is there a version of arithmetic coding that provides worst-case local random access during decoding, perhaps by combining the paper’s two-dimensional encoding with the interval-refinement view? The current paper does not address this, and no subsequent work known to this reviewer resolves it in full generality.

- (3) **Richer succinct structures.** Can the local base-conversion technique be lifted to structures that support rank, select, or dictionary operations? The paper separates plain coordinate access from these richer queries, but an extension that handles even a restricted subset of rank queries would be a significant advance.
- (4) **Dictionary and rank/select.** Yu’s dictionary [20] uses fractional-length strings from spillover representations, which share a conceptual similarity with information carriers. An explicit unification of these two fractional-information viewpoints—information carriers from this paper and spillover representations from Pătraşcu’s “Succincter”—could clarify the design space for succinct dictionaries.
- (5) **Reducing division overhead.** Can the constant number of divisions and multiplications per carrier be reduced, perhaps by using precomputed reciprocals or alternative algebraic encodings? This would narrow the gap between the Word RAM cost model and practical implementations.
- (6) **Toward the Elias iterated-log bound online.** The online code achieves $n + \log_2 n + O(\log \log n)$, while the offline Elias omega code achieves $n + \log_2 n + \log_2 \log_2 n + \dots + O(\log^* n)$. Is an online code with the full iterated-log expression possible under the same $O(\log n)$ -memory and constant-word-time constraints? The current paper does not claim optimality in this direction.
- (7) **Unified framework.** The post-2010 work on fractional-length strings and spillover representations [20] and the present paper’s information carriers address different problems with related primitives. A unified framework that captures both as instances of a common fractional-information calculus would be valuable for theory, even if it does not immediately yield new upper bounds.

10 Conclusion

10.1 Summary

Changing Base without Losing Space addresses two constrained base-conversion problems whose difficulty comes from a single source: data naturally belongs to ranges whose cardinalities are not powers of two, while binary memory is. The paper’s two main results are an exact-space vector representation that supports $O(1)$ reads and writes, and a low-memory online prefix-free code whose length is within $O(\log \log n)$ of the Elias-scale optimum.

10.2 The Unifying Role of Information Carriers

The common technical core is the information-carrier lemma (Lemma 1). It shows that a sufficiently large carrier value can commit an old spill to binary memory while producing a new spill whose range is only $O(\sqrt{X})$. The decoding guarantee is asymmetric in a crucial way: the old spill is recoverable from the memory content alone, without accessing the new spill. This single condition is what keeps queries and updates local.

The two main results are applications of the same lemma with different organizational structures. Placing carriers in an implicit binary tree yields a vector data structure with $O(1)$ -time reads and writes, exact space, and $O(\log n)$ precomputed constants. Arranging

carriers in a sequential chain with slowly growing block sizes yields an online prefix-free code with logarithmic working memory and near-optimal redundancy.

10.3 What Makes the Paper Surprising

The paper’s most counter-intuitive contribution is the diagnosis that the space–locality tension comes from integral-bit rounding, not from information theory or lower bounds. One usually thinks of information as the fundamental resource; here the real resource being wasted is the unused fraction of an integer bit. Because the waste is structural rather than information-theoretic, it can be eliminated by structural means—local, reversible base conversion—without violating any known lower bound.

The second surprise is that locality can be enforced by a purely syntactic decoding asymmetry. Requiring that the old spill be extractable from the memory word alone is simple to state, easy to verify in the construction, and yet sufficient to ensure that no chain of dependencies can develop across carriers.

10.4 Theoretical Value

The paper makes a clean theoretical contribution. It identifies a specific source of overhead, isolates it in a lemma with explicitly bounded redundancy, and demonstrates two different ways to organize carriers so that the accumulated overhead is either $o(1)$ (vector) or $O(\log \log n)$ (stream). The constructions are explicit, the bounds are stated with concrete asymptotic parameters, and the proof obligations are clearly decomposed.

The vector result is, within its model, definitive: no representation of $A[1..n]$ over Σ can use fewer than $\lceil n \log_2 \Sigma \rceil$ bits, and this representation achieves that bound exactly while supporting $O(1)$ reads and writes. The online encoding result is near-optimal within $O(\log \log n)$ and refutes a natural conjecture about the cost of online prefix-freeness.

10.5 Practical Applicability

The results are stated in the Word RAM model with unit-cost multiplication and division. Their translation to practical performance depends on the concrete word size, division latency, cache organization, and the overhead of precomputed constants. The paper does not provide, and does not claim to provide, implementation benchmarks or evidence of deployment. The contribution is primarily to the theoretical understanding of what is possible, not to a specific engineering artifact.

The uniform-alphabet assumption also limits direct applicability. Many practical compression and storage systems deal with skewed distributions, variable-length objects, or richer query requirements. The paper’s techniques do not directly address those settings, although the underlying insight—that fractional information can be accumulated and committed locally—may inform future work in those directions.

10.6 Final Assessment

Changing Base without Losing Space is a crisp, self-contained paper that solves two well-motivated problems with a single, elegant primitive. Its impact is primarily conceptual: it shows that the integral-bit barrier is avoidable for uniform finite alphabets, that round-by-round local conversion can eliminate redundancy, and that one asymmetric decoding condition suffices to guarantee locality across two quite different organizational schemes.

The paper does not claim to solve all of succinct data structures, and it should not be assessed as if it did. Within its precisely stated model and problem scope, the results are complete, the analysis is rigorous, and the exposition builds from a concrete example (SOLE) to an abstract lemma (information carrier) to two clean theorems. Its limitations—the Word RAM assumptions, the uniform-alphabet restriction, and the absence of implementation evidence—are acknowledged or clearly visible from the problem statement. The paper remains a noteworthy contribution to the theory of succinct representation and a thought-provoking demonstration that changing base need not cost space.

References

- [1] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In *Advances in Cryptology—CRYPTO '96*, pages 1–15. Springer, 1996.
- [2] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Pseudorandom functions revisited: The cascade construction and its concrete security. In *Proceedings of the 37th IEEE Symposium on Foundations of Computer Science*, pages 514–523. IEEE, 1996.
- [3] Mihir Bellare, Joe Kilian, and Phillip Rogaway. The security of cipher block chaining. In *Advances in Cryptology—CRYPTO '94*, pages 341–358. Springer, 1994.
- [4] Mihir Bellare and Thomas Ristenpart. Multi-property-preserving hash domain extension and the EMD transform. In *Advances in Cryptology—ASIACRYPT 2006*, pages 299–314. Springer, 2006.
- [5] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. Merkle–Damgård revisited: How to construct a hash function. In *Advances in Cryptology—CRYPTO 2005*, pages 430–448. Springer, 2005.
- [6] Yevgeniy Dodis, Mihai Pătraşcu, and Mikkel Thorup. Changing base without losing space. In *Proceedings of the 42nd ACM Symposium on Theory of Computing*, pages 593–602. ACM, 2010.
- [7] Peter Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2):194–203, 1975.
- [8] Anna Gál and Peter Bro Miltersen. The cell probe complexity of succinct data structures. In *Proceedings of the 30th International Colloquium on Automata, Languages and Programming*, pages 332–344. Springer, 2003.
- [9] Oded Goldreich, Shafi Goldwasser, and Silvio Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986.
- [10] Alexander Golynski. Cell probe lower bounds for succinct data structures. In *Proceedings of the 20th ACM-SIAM Symposium on Discrete Algorithms*, pages 625–634. SIAM, 2009.
- [11] Alexander Golynski, Roberto Grossi, Ankur Gupta, Rajeev Raman, and S. Srinivasa Rao. On the size of succinct indices. In *Proceedings of the 15th European Symposium on Algorithms*, pages 371–382. Springer, 2007.

- [12] Guy Jacobson. Space-efficient static trees and graphs. In *Proceedings of the 30th IEEE Symposium on Foundations of Computer Science*, pages 549–554. IEEE, 1989.
- [13] Ueli M. Maurer and Johan Sjödin. Single-key AIL-MACs from any FIL-MAC. In *Proceedings of the 32nd International Colloquium on Automata, Languages and Programming*, pages 472–484. Springer, 2005.
- [14] Alistair Moffat, Radford M. Neal, and Ian H. Witten. Arithmetic coding revisited. *ACM Transactions on Information Systems*, 16(3):256–294, 1998.
- [15] Mihai Pătraşcu. Succincter. In *Proceedings of the 49th IEEE Symposium on Foundations of Computer Science*, pages 305–313. IEEE, 2008.
- [16] Mihai Pătraşcu and Emanuele Viola. Cell-probe lower bounds for succinct partial sums. In *Proceedings of the 21st ACM-SIAM Symposium on Discrete Algorithms*, pages 117–122. SIAM, 2010.
- [17] Emanuele Viola. Bit-probe lower bounds for succinct data structures. In *Proceedings of the 41st ACM Symposium on Theory of Computing*, pages 475–482. ACM, 2009.
- [18] Ian H. Witten, Radford M. Neal, and John G. Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540, 1987.
- [19] Huacheng Yu. Optimal succinct rank data structure via approximate nonnegative tensor decomposition. In *Proceedings of the 51st ACM Symposium on Theory of Computing*, pages 955–966. ACM, 2019.
- [20] Huacheng Yu. Nearly optimal static Las Vegas succinct dictionary. *SIAM Journal on Computing*, 51(3):20–174, 2022.